# Triton Optimization Pass Lab

Explore compiler optimizations from a Triton kernel through TTIR, TTGIR, LLVM IR, and PTX.

Main example: blocked GEMM

Comparison example: fused softmax

Key topics: data layout, software pipelining, tensor core lowering

# Lab Deliverables

- kernels.py: GEMM and softmax kernels
- run_lab.py: executes kernels and exports IR
- artifacts/: TTIR, TTGIR, LLVM IR/LLIR, PTX
- README.md: instructions and IR reading guide
- slides.typ: lecture slides

Recommended command:

```
python labs/triton-opt/run_lab.py --kernel all
```

# The Compilation Pipeline at a Glance

```
Python Triton kernel
  -> TTIR
  -> TTIR canonicalization / CSE / LICM
  -> TTGIR with GPU layout encodings
  -> layout propagation / dot operand optimization
  -> software pipelining / prefetch
  -> LLVM IR or LLIR
  -> PTX with mma.sync or wgmma
```

Key question: What information is added at each IR level, and what high-level semantics are lost?

# Why Use GEMM as the Main Example?

GEMM covers several of the most important optimization topics in the Triton compiler:

- The K-dimension loop is ideal for explaining software pipelining
- tl.dot is a high-level anchor for tensor core lowering
- A/B/C tile layouts directly affect memory and compute efficiency
- Shared-memory swizzling, bank conflicts, and MMA operand layouts all leave traces in the IR

# GEMM Kernel Structure

```
pid = tl.program_id(axis=0)
offs_m = ...
offs_n = ...
acc = tl.zeros((BLOCK_M, BLOCK_N), tl.float32)

for k_start in range(0, K, BLOCK_K):
    a = tl.load(...)
    b = tl.load(...)
    acc += tl.dot(a, b)

tl.store(..., acc)
```

This code maps naturally to program mapping, masked loads, dot products, loop-carried accumulators, and stores.

# Why Compare Against Softmax?

Softmax does not emphasize tensor cores, but is excellent for examining TTIR:

- One program per row
- Masks handle column counts that are not powers of two
- load -> max -> exp -> sum -> div -> store are fused in one kernel
- Reductions and elementwise operations mix in a shorter IR than GEMM

It helps students see that Triton optimization goes beyond matrix multiplication.

# TTIR: Close to Triton Semantics

Look for the following in 00_ttir.mlir:

- tt.program_id
- tt.load and tt.store
- tt.dot
- tt.exp, tt.reduce, or a reduction region
- Masks and pointer arithmetic
- scf.for or an unrolled K-dimension loop

TTIR still reveals what the programmer originally wrote.

# Optimizations at the TTIR Level

TTIR primarily supports general compiler optimizations:

- Canonicalization: normalize the IR into easier-to-match forms
- CSE: eliminate redundant expressions
- LICM: hoist loop-invariant computations
- DCE: remove dead computations
- Inlining / symbol cleanup: simplify JIT-generated function boundaries

In class: the final TTIR is an optimized snapshot, not necessarily a before-and-after view of each pass.

# TTGIR: Adding GPU Layouts

The key change in TTGIR: layout encodings become part of tensor types.

Common clues:

- `#ttg.blocked`
- `#ttg.dot_op`
- `#ttg.nvidia_mma`
- Shared-memory layout encodings
- `ttg.convert_layout`

TTGIR asks not only how large a tensor is, but also how its elements are distributed across CTAs, warps, threads, registers, and shared memory.

# Intuition Behind Data Layout

The same tile can have many physical arrangements:

```
logical tile:
  [M x N]


physical questions:
  which CTA owns it?
  which warp owns each slice?
  which lane owns each element?
  is it in registers or shared memory?
  does shared memory need swizzle?
```

Layout is the key bridge from high-level tensor programs to GPU hardware in Triton.

# Why Is convert_layout Expensive?

ttg.convert_layout is more than a change in type annotation.

It may involve:

- Warp shuffles
- Transfers through shared memory
- Register rearrangement
- Extra barriers or waits
- Longer live ranges and higher register pressure

Therefore, TTGIR applies layout propagation, conversion removal, and rematerialization.

# Layout Propagation

A simplified model:

```
anchor ops choose preferred layout
  load/store prefer memory-friendly blocked layout
  dot prefers MMA-friendly operand layout

propagate layout through cheap ops
resolve conflicts
rewrite IR
remove redundant conversions
```

Class question: If load and dot prefer different layouts, where should the compiler convert between them?

# Dot Operand Optimization

The most important anchor in GEMM is tl.dot.

TTGIR performs the following around dot:

- Convert inputs to dot operand encodings
- Select an MMA accumulator layout for tensor cores
- Place some operands in shared memory
- Use swizzling to reduce bank conflicts
- On the Hopper path, fold transposes into memory descriptors

# Shared Memory Swizzling

Why does shared memory need swizzling?

```
bad layout:
  many lanes hit the same bank
  -> bank conflict
  -> serialized access

swizzled layout:
  lanes spread across banks
  -> higher bandwidth
  -> tensor core operands arrive faster
```

IR clues: shared-memory allocation, local loads, and dot operand layouts.

# Software Pipelining

Goal: overlap memory access with computation.

```
no pipeline:
  load k0 -> dot k0 -> load k1 -> dot k1


pipeline:
  preload k0, k1
  dot k0 while loading k2
  dot k1 while loading k3
```

Adjust --num-stages in the script to change the pipeline depth.

# The Three Phases of a Pipeline

Software pipelining typically generates:

- Prologue: prefill several stages before the main loop
- Steady-state kernel: interleave operations from multiple iterations
- Epilogue: finish the remaining stages after the main loop

IR clues:

- New loop arguments
- Cross-stage values
- Circular-buffer indices
- Async copy / wait / barrier

# The num_stages Trade-off

More stages may hide more latency, but more is not always better.

Costs:

- Higher shared-memory usage
- Longer register live ranges
- Potentially lower occupancy
- More complex barriers and scheduling

Experiment: Do IR changes and performance changes agree for --num-stages 1/3/5?

# Tensor Core Lowering

From high-level operations to hardware:

```
tl.dot
  -> tt.dot
  -> TTGIR dot with MMA layouts
  -> NVIDIA MMA lowering
  -> LLVM / NVVM
  -> PTX instruction
```

Ampere / Ada: mma.sync is common

Hopper: you may see wgmma.mma_async and waits/barriers

# What to Look For in PTX

Search keywords:

- `mma.sync`
- `wgmma`
- `ldmatrix`
- `barrier`
- `wait`
- `.shared`

Map these back to high-level concepts:

tl.dot, shared-memory tiles, pipeline stages, async waits, and layout conversions.

# Student Lab Tasks

Four suggested comparisons:

- Change BLOCK_M/N/K: How do tile shapes and layouts change?
- Change num_warps: How does warp distribution change?
- Change num_stages: How does the pipeline structure change?
- Compare matmul and softmax: Why do the optimization priorities differ?

Ask students to show IR screenshots or excerpts and explain the corresponding passes.

# Summary

Triton optimization can be understood in layers:

- TTIR: high-level tensor programs and general optimizations
- TTGIR: GPU layouts, dot operands, shared memory, and pipelines
- LLVM/PTX: hardware instructions, synchronization, registers, and memory details

Key takeaway: Triton performance comes from the combination of its programming model, layout system, and hardware lowering.